

Ridge Regression with Gradient Descent

Contents

1. Overview	2
2. The gradient	2
2.1 The loss with a factor of $\frac{1}{2}$	2
2.2 Expanding and differentiating	3
2.3 The update rule	3
3. Seeing the paths	3
4. Code from scratch	4
5. Gradient descent stops early	5
6. Ridge with gradient descent in scikit-learn	7
7. Summary	8
8. Sources	8
9. Key terms	8

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Standardization ([Note 24](#)); Multiple linear regression ([Note 55](#)); Normal equation ([Note 55](#)); Gradient descent ([Note 57](#)); Bias-variance trade-off ([Note 62](#)); Regularisation ([Note 63](#)).
- **Leads to:** Elastic Net ([Note 69](#)).
- **Compare with:** Lasso regression ([Note 67](#)); L1 and L2 regularisation in neural networks ([Note 1026](#)).

1. Overview

Key point: Ridge can also be trained with gradient descent. The gradient is the linear regression gradient plus λw , so each step also pulls the coefficients a little towards 0.

The previous Note found the Ridge coefficients in one step with a formula. Like plain linear regression, Ridge can instead be trained with **gradient descent**: start somewhere, and repeatedly step downhill on the loss.

Gradient descent is useful when there are many input columns, because the formula needs the inverse of a large matrix (the gradient descent Notes). This Note derives the Ridge gradient, codes it from scratch, and shows the scikit-learn options.

2. The gradient

Key point: With a factor of $\frac{1}{2}$ in the loss, the gradient is $X^T X w - X^T y + \lambda w$.

2.1 The loss with a factor of $\frac{1}{2}$

Key point: Multiplying the loss by $\frac{1}{2}$ does not move its minimum; it only removes a 2 from the derivative.

The Ridge loss from the previous Note, multiplied by $\frac{1}{2}$:

$$L = \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{1}{2}\lambda w^T w$$

Halving every value of the loss leaves the lowest point at the same w . It just makes the derivative tidier, as the 2 from differentiating a square cancels the $\frac{1}{2}$.

2.2 Expanding and differentiating

Key point: The same expansion as the previous Note, then the same derivative rules.

Expanding the product as in the normal equation Note:

$$L = \frac{1}{2} \left(w^\top X^\top X w - 2w^\top X^\top y + y^\top y \right) + \frac{1}{2} \lambda w^\top w$$

Differentiating with respect to w :

$$\frac{\partial L}{\partial w} = X^\top X w - X^\top y + \lambda w$$

The first two terms are the linear regression gradient. The Ridge penalty adds only λw .

Extra: Setting this gradient to zero gives $(X^\top X + \lambda I)w = X^\top y$, the formula of the previous Note. Gradient descent walks towards the same answer step by step instead of jumping to it.

2.3 The update rule

Key point: New w = old w minus the learning rate times the gradient.

As in every gradient descent Note, all coefficients are updated together:

$$w_{\text{new}} = w_{\text{old}} - \eta \left(X^\top X w_{\text{old}} - X^\top y + \lambda w_{\text{old}} \right)$$

where η is the learning rate. The first entry of w is the intercept, which is not penalised. So, as before, the λw term uses $\lambda I w$ with the top-left entry of I set to 0.

Extra: Rearranging the update gives $w_{\text{new}} = (1 - \eta\lambda) w_{\text{old}} - \eta (X^\top X w_{\text{old}} - X^\top y)$. So every step first shrinks the coefficients by the factor $1 - \eta\lambda$, then takes the ordinary linear regression step. This is why the L2 penalty is also called **weight decay**, the name used for neural networks (ESL §3.4.1).

3. Seeing the paths

Key point: Gradient descent finds the bottom of whichever loss it is given. With $\lambda = 100$, the bottom sits at a smaller slope.

Figure 1 runs 60 steps of gradient descent on the one-input example of the previous Notes, from the same start point ($m = -5$, $b = 20$, learning rate 0.005).

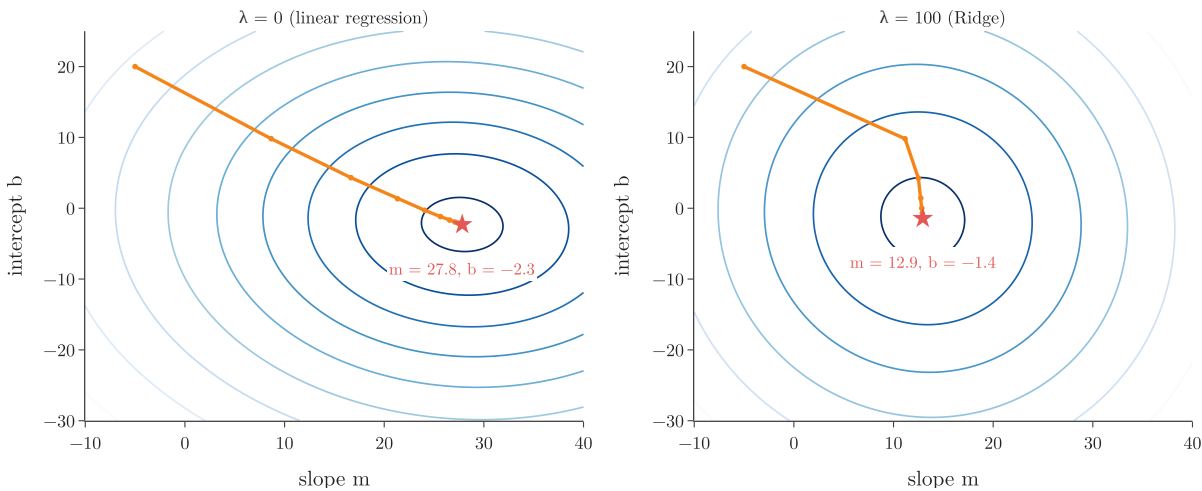


Figure 1: Gradient descent on the loss with $\lambda = 0$ and $\lambda = 100$

- $\lambda = 0$: the path ends at $m = 27.8$, $b = -2.3$, the linear regression answer.
- $\lambda = 100$: the penalty moves the bottom of the bowl to $m = 12.9$, $b = -1.4$. These are exactly the Ridge values from the formula in the previous Note.

The penalty makes the bowl steeper in the m direction only: its curvature there, $\partial^2 L / \partial m^2 = \sum x_i^2 + \lambda$, grows from 87 to 187, while the b direction is unchanged.

4. Code from scratch

Key point: The loop computes the gradient with matrix products and updates all coefficients at once.

Python: Ridge with batch gradient descent.

```
class MyRidgeGD:
    def __init__(self, epochs, learning_rate, alpha):
        self.epochs, self.lr, self.alpha = (epochs,
                                             learning_rate, alpha)

    def fit(self, X, y):
        X = np.insert(X, 0, 1, axis=1)           # column of 1s
        w = np.insert(np.ones(X.shape[1] - 1), 0, 0.0)
        I = np.identity(X.shape[1])
        I[0, 0] = 0                               # no penalty on intercept
        for _ in range(self.epochs):
            grad = X.T @ X @ w - X.T @ y + self.alpha * I @ w
            w = w - self.lr * grad
        self.intercept_, self.coef_ = w[0], w[1:]
        return self
```

```
def predict(self, X):
    return X @ self.coef_ + self.intercept_
```

On the diabetes data (test size 0.2, random state 4) with alpha 0.001, learning rate 0.005 and 500 epochs, test R^2 is 0.474 and the intercept 150.87.

Extra: This gradient adds up the errors of all rows instead of averaging them, so its size grows with the number of rows. That is why the learning rate must be tiny here. The maths: write $H = X^T X + \lambda I$ and w^* for the exact answer. One update turns the error $w - w^*$ into $(I - \eta H)(w - w^*)$. Along each eigenvector of H with eigenvalue h , the error is multiplied by $1 - \eta h$ every step, so it shrinks only if $\eta < 2/h$ for the largest h . On this training set (353 rows) the largest eigenvalue is 353, so the limit is $\eta < 2/353 = 0.0057$. 0.005 is just below it; 0.006 is just above it, and the steps overshoot and grow: after 100 epochs the largest coefficient is about 15,000.

5. Gradient descent stops early

Key point: After 500 epochs, gradient descent is still far from the exact answer. Stopping early keeps the coefficients small, so early stopping is a regulariser of its own, much like the ridge penalty.

The exact Ridge answer from the previous Note scores test R^2 0.463 here. Figure 2 tracks gradient descent for up to a million epochs.

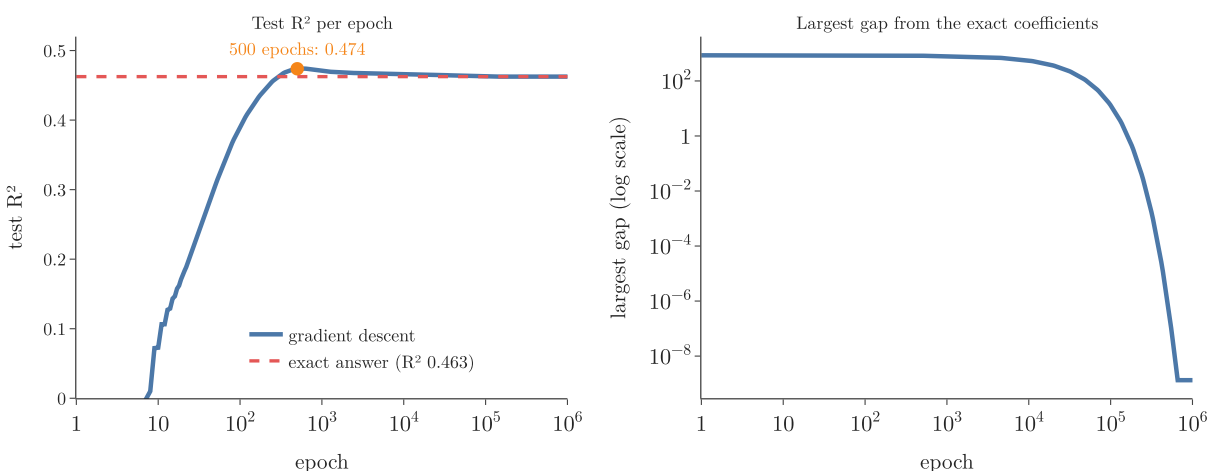


Figure 2: Test R^2 and distance from the exact answer, per epoch

Epochs	Test R^2	Largest gap from the exact coefficients
100	0.390	897

Epochs	Test R^2	Largest gap from the exact coefficients
500	0.474	823
50,000	0.464	104
1,000,000	0.463	0.0

- **Left:** test R^2 rises, peaks around 500 epochs at 0.474, then drifts down to the exact answer's 0.463.
- **Right:** the coefficients need about a million epochs to reach the exact values.

Why so slow? s_1 and s_2 are strongly correlated (the [first Ridge Note](#)), so the bowl is almost flat along the direction “raise s_2 , lower s_1 ”, and gradient descent creeps along it. Stopping early leaves the coefficients in that direction smaller, which acts like extra regularisation (Goodfellow et al. §7.8).

When does that extra regularisation pay off? Only when the plain least squares fit overfits. With the 10 diabetes features (input variables, the columns of the data table) and 353 training observations (patients, the rows) it barely does, so the small rise from 0.463 to 0.474 on this one split means little. Shrinkage helps most when the least squares fit has high variance, for example with many features for few observations (ISL §6.2.1).

To see it pay off, we change one thing: we give the model 65 features (the 10 measurements plus their squares and pairwise products) and train on 200 patients, as in the [batch gradient descent Note](#) (section five). Averaged over 50 random splits, test R^2 is:

Model	Test R^2
Linear regression (no regularisation)	0.06
Ridge, alpha chosen by cross-validation on the training data	0.44
Gradient descent without a penalty, stopped early	0.40

Both regularisers rescue the badly overfitting linear regression, and by a similar amount. For a linear model, stopping early and adding the ridge penalty are close cousins (Goodfellow et al. §7.8).

Extra: The numbers behind this. Along an eigenvector of $H = X^T X + \lambda I$ with eigenvalue h , the error shrinks by $1 - \eta h$ per step, so it needs about $1/(\eta h)$ steps. The smallest eigenvalue here is 0.0083, giving about $1/(0.005 \times 0.0083) \approx 24,000$ steps, and its eigenvector is mostly s_1 (-0.71) and s_2 ($+0.55$). After 500 epochs, almost the whole gap from the exact answer (1,146 of it) lies along this one direction, and the coefficient vector is about half as long as the exact one (774 against 1,455).

Extra: In the 65-feature experiment both models use standardised features. The stopping epoch is chosen on a validation part of the training data, and ridge's alpha by cross-validation on the training data, so the test data is used only once. Early stopping is one of the most common regularisers in deep learning (Goodfellow et al. §7.8).

6. Ridge with gradient descent in scikit-learn

Key point: Use SGDRegressor with penalty="l2", or Ridge with an iterative solver.

Python: Two scikit-learn options.

```
from sklearn.linear_model import SGDRegressor, Ridge

sgd = SGDRegressor(penalty="l2", alpha=0.001, max_iter=500,
                   eta0=0.1, learning_rate="constant",
                   random_state=0)
sgd.fit(X_train, y_train)
sgd.score(X_test, y_test)           # R2 0.450

ridge = Ridge(alpha=0.001, max_iter=500, solver="sparse_cg")
ridge.fit(X_train, y_train)
ridge.score(X_test, y_test)        # R2 0.463
```

- **SGDRegressor** (the stochastic gradient descent Note) adds the Ridge penalty with penalty="l2". Its alpha is the penalty strength.
- **Ridge** has a solver setting. "cholesky" and "svd" use the formula directly. "sparse_cg", "lsqr", "sag" and "saga" reach the answer step by step, and max_iter limits the steps.

Ridge has no learning-rate setting at all, even with an iterative solver (scikit-learn docs, Ridge). Here sparse_cg reaches the exact answer, 0.463.

Extra: SGDRegressor averages the loss over the rows, while Ridge adds it up. The SGDRegressor objective is (scikit-learn user guide §1.5.8) $\frac{1}{n} \sum \frac{1}{2}(y_i - \hat{y}_i)^2 + \alpha \cdot \frac{1}{2}\|w\|^2$. Multiplying by $2n$ gives $\sum (y_i - \hat{y}_i)^2 + n\alpha\|w\|^2$, the Ridge loss with alpha $n\alpha$. So the same alpha means a stronger penalty in SGDRegressor: its alpha equals Ridge's alpha divided by the number of rows. With 353 training rows, SGDRegressor(alpha=0.001) matches Ridge(alpha=0.353): run long, its coefficients land within 3 of that model's, but up to 840 away from Ridge(alpha=0.001).

7. Summary

Method	How it finds w	Test R^2 here
Formula (<code>Ridge</code> , <code>cholesky</code>)	one step, $(X^T X + \lambda I)^{-1} X^T y$	0.463
<code>Ridge</code> , <code>sparse_cg</code>	iterative solver	0.463
<code>MyRidgeGD</code> , 500 epochs	batch gradient descent, stopped early	0.474
<code>SGDRegressor</code> , <code>l2</code>	stochastic gradient descent	0.450

- The Ridge gradient is $X^T X w - X^T y + \lambda w$: the linear regression gradient plus λw .
- Each update shrinks the coefficients by $1 - \eta \lambda$ first, hence the name weight decay.
- The intercept is left out of the penalty.
- Stopping gradient descent early also regularises: with many features for few observations it rescues an overfitting fit about as well as ridge.
- In scikit-learn: `SGDRegressor(penalty="l2")` or `Ridge` with an iterative solver.

8. Sources

- **ESL:** Hastie, T., Tibshirani, R. and Friedman, J. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. Section 3.4.1, p. 63.
- **Goodfellow et al.:** Goodfellow, I., Bengio, Y. and Courville, A. *Deep Learning*. MIT Press, 2016. Section 7.8, “Early Stopping”.
- **scikit-learn docs:** `sklearn.linear_model.Ridge`; user guide Section 1.5.8, “Mathematical formulation” (SGD), scikit-learn 1.9.
- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021. Section 6.2.1, p. 241.

9. Key terms

Term	Meaning
Weight decay	Another name for the L2 penalty: each gradient step shrinks the coefficients by a fixed factor
Solver	The method a scikit-learn model uses to find its coefficients
penalty	<code>SGDRegressor</code> setting that adds a regularisation penalty, such as “l2” for <code>Ridge</code>